

# Parcial Funcional 2026 — Meeseeks destructores

## Tema 1

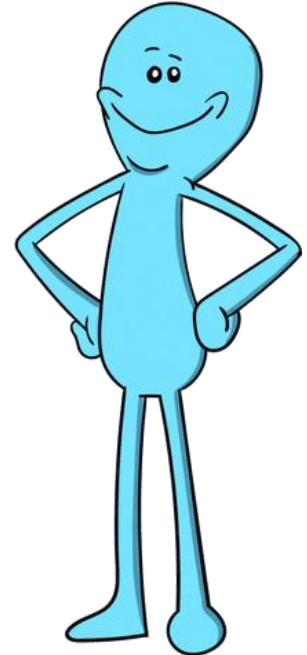
*"I'm Mr. Meeseeks! Look at me!"*

Rick le regaló a la familia Smith una Caja de Meeseeks: un dispositivo que, al apretar el botón, hace aparecer un Mr. Meeseeks con la tarea que se le pida. Los Meeseeks existen para servir: aparecen, cumplen su tarea, y desaparecen. Si la tarea se demora, sufren cada vez más, se vuelven más oscuros y peligrosos.

De cada Meeseeks conocemos:

- su nombre (siempre arranca con "Mr. Meeseeks", pero puede recibir sufijos),
- el color, que va del más claro al más oscuro: "celeste", "azul", "violeta", "negro",
- las horas de existencia que lleva activo (un número),
- la tarea actual que está intentando cumplir (por ejemplo, "sacar dos golpes al golf"),
- el historial de intentos: una colección de intentos, donde cada intento tiene una descripción de la tarea y la cantidad de horas que le tomó.

Una caja es un conjunto de Meeseeks.



### Punto 1: I'm Mr. Meeseeks! (3 puntos)

#### a) Modelar los Meeseeks

Según lo indicado en la introducción. Adicionalmente, se pide modelar un Meeseeks de ejemplo.

#### b) Color que merece

Queremos saber el color que le corresponde a un Meeseeks según sus horas de existencia:

- Si lleva menos de 1 hora y si el nombre tiene longitud par, le corresponde "celeste".
- Si lleva entre 1 y 5 horas (inclusive), le corresponde "azul".
- Si lleva más de 5 y hasta 24 horas, le corresponde "violeta".
- Si lleva más de 24 horas, le corresponde "negro" (entró en pánico existencial).

#### c) ¿Está sufriendo?

Queremos saber si un Meeseeks está sufriendo, lo cual ocurre si su color actual no coincide con el color que le correspondería según sus horas de existencia, o si su tarea actual contiene la palabra "golf" (los Meeseeks odian el golf). Para este parcial, se busca exactamente la palabra "golf" en minúscula.

# Parcial Funcional 2026 — Meeseeks destructores

## Punto 2: Look at me! (2 puntos)

*En este punto no puede utilizar funciones auxiliares ni recursividad, solo composición, aplicación parcial y funciones de orden superior.*

### a) Caja agotadora

Queremos saber si una caja es agotadora, esto pasa cuando todos sus Meeseeks tienen al menos un intento en su historial que les tomó más de una cantidad de horas configurable. Por ejemplo, si la cantidad es 10, todos los Meeseeks deben tener al menos un intento con más de 10 horas.

### b) Nombres de sufridos persistentes

Queremos obtener los nombres de los Meeseeks de la caja que están sufriendo y que tienen más de una determinada cantidad de intentos en su historial.

### c) Justifique

Indique en al menos una parte del código realizado dónde utilizó composición, aplicación parcial y orden superior.

## Punto 3: Can do! (2,5 puntos)

*En este punto no debe repetir lógica*

Existen distintas interacciones que la familia puede tener con un Meeseeks. Cada interacción transforma al Meeseeks de alguna manera:

- **Pedir ayuda:** le suma 1 hora de existencia al Meeseeks y le cambia la tarea actual por la nueva tarea pedida.
- **Golpear:** enloquece al Meeseeks. Le agrega el sufijo " (enojado)" al nombre y duplica sus horas de existencia. Si no tenía horas (recién salió de la caja), lo deja con 1 hora para que sienta algo.
- **Agradecer:** le permite al Meeseeks completar su tarea. Esto agrega al historial un intento con la tarea actual y las horas de existencia que llevaba, reinicia las horas de existencia en 0 y la tarea actual en "ninguna".
- **Frustrarse:** como hacen los Meeseeks cuando otro Meeseeks no puede ayudarlos. Hace lo mismo que golpear, pero el doble de veces.

Indicar cómo se haría en la consola para que un Meeseeks reciba la siguiente secuencia de interacciones: que le pidan ayuda con la tarea "abrir un frasco", que sea golpeado, que se frustre y que finalmente le agradezcan. No importa el resultado, sino cómo es la invocación en el REPL.

# Parcial Funcional 2026 — Meeseeks destructores

## Punto 4: Existence is pain! (2,5 puntos)

*En este punto debe utilizar recursividad. Adicionalmente, maximizar el uso de los demás conceptos vistos en la materia (composición, aplicación parcial, orden superior, etc.).*

La familia descubrió que apretando el botón de la caja una y otra vez puede llevar a todos los Meeseeks al colapso.

Apretar el botón, significa aplicar a cada Meeseeks de la caja, una secuencia de interacciones negativas (los castigos) que se recibe como parámetro. Las interacciones se aplican en orden: la primera de la secuencia se aplica primero. Por ejemplo, con los castigos [pedirAyuda "limpiar el sótano", golpear, frustrarse], a cada Meeseeks primero se le pide ayuda, después se lo golpea y después se frustra.

### a) Hasta colapsar

Queremos una función que, dada una secuencia de castigos y una caja, se apriete el botón repetidamente sobre toda la caja hasta que todos los Meeseeks estén sufriendo, y devuelva la caja en ese estado de colapso.

Por ejemplo en una primera pasada apretamos el botón y aplicamos los castigos sobre toda la caja pero aún no están todos sufriendo, por lo tanto se vuelve a apretar el botón a toda la caja para aplicar los castigos a todos los Meeseeks nuevamente. Esta operatoria se repite hasta que todos se encuentren sufriendo.

Ojo: acá hay dos cosas distintas. Por un lado, aplicarle a cada Meeseeks las interacciones de la secuencia de castigos, una tras otra. Por otro, repetir el *botoneo* sobre la caja una y otra vez hasta que todos sufran. Pensá cuál de las dos es la que realmente necesita recursividad.

### b) ¿Siempre colapsan?

¿La función del punto anterior siempre termina si la lista de castigos es infinita? . ¿De qué depende que la función termine o no? Relacione su respuesta con el concepto de condición de corte de un algoritmo recursivo